

Accelbench getting started guide

Introduction

Accelbench is a platform for benchmarking open-source large language models (LLMs) on different types of accelerated EC2 instances. During a benchmarking run, key metrics are collected, such as latency, throughput, hardware utilization, and cost. These metrics are frequently used by practitioners to choose the instances type to run a particular model on. Oftentimes, practitioners need to strike a balance between performance, cost, and accuracy (not currently measured by Accelbench). For example, a model may perform extremely well on a particular instance type, but the cost of running that instance is too high or you're not able to saturate the instance's accelerator. Accelbench gives practitioners a simple, intuitive way to collect these key performance benchmarks and do side-by-side comparisons of the results, giving them the information they need to quickly choose an instance type that meets their performance/cost requirements and constraints.

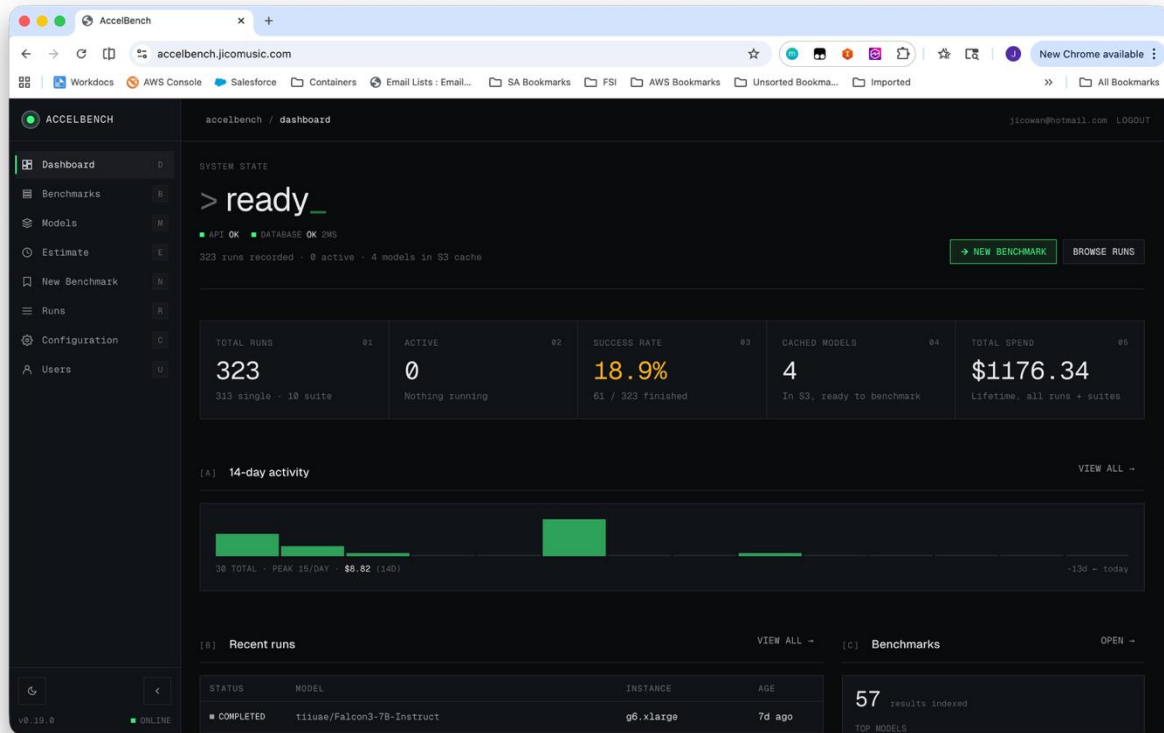
The interface

The interface is divided into 8 major sections: Dashboard, Benchmarks, Models, Estimate, New Benchmark, Runs, Configuration, and Users. These sections can be categorized into 2 major categories: end-user and administrator. Let's start by looking at the end-user sections, beginning with the **Dashboard**.

End-user guide

The Dashboard

From the **Dashboard**, you can see a high-level overview of the activity that has occurred on the platform along with the platform's status. The Dashboard reveals how many benchmarking runs have occurred (single scenarios and test suites), the number of active runs, the overall success rate, the number of cached models, and the total compute cost for running the benchmarks. It also shows a 14-day trailing history of the activity that has occurred on the platform, including the number of runs per day and the daily cost.

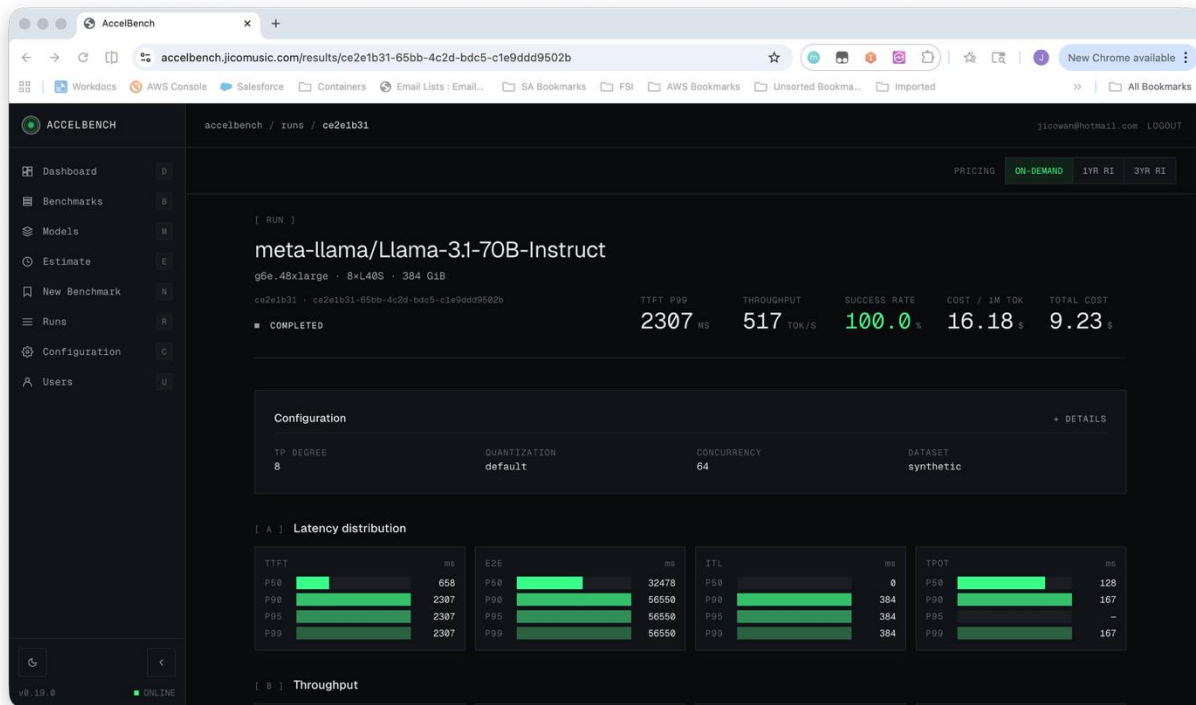


From this page, you can start a new benchmark or browse completed benchmarking runs.

Benchmarks

From the **Benchmarks** page you will see the list of completed benchmarks and their key metrics. Only completed single scenario runs appear in the **Benchmarks** catalog. Test suite runs, i.e. benchmark runs that are comprised of multiple scenarios, will appear on the **Runs** page. Clicking on an individual benchmark will open the full benchmarking report where you can view the latency distribution (P50-P99), throughput and hardware utilization statistics, and costs.

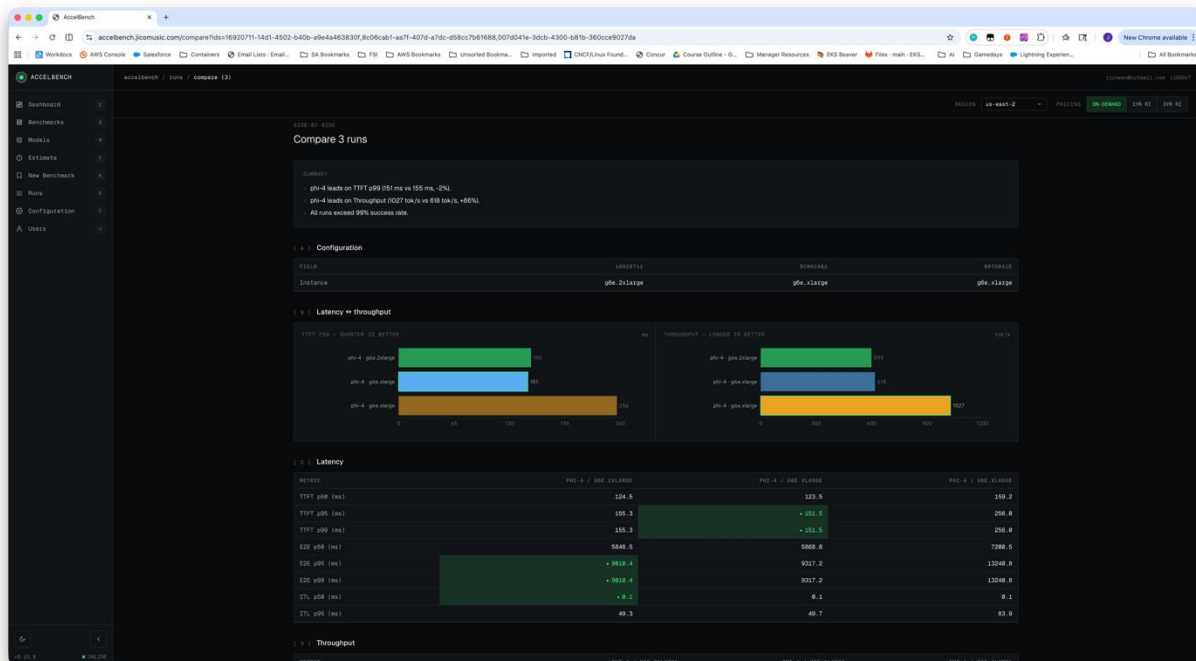
By default, Accelbench calculates the price based on the the on-demand cost for the instance. The selectors at the top of the page allow you to toggle between on-demand, 1 year RI, and 3 year RI costs. Calculating the cost based on custom discounts, ODCRs, capacity blocks, and Spot pricing is not currently supported.



All reports can be printed and exported to a CSV file. The buttons for doing so appear at the bottom of the single scenario, and suite reports. You can also export/download Kubernetes manifest that was used for an individual benchmark run. This allows you easily run the model in another environment if you so choose.

The comparison report allows you to see a side-by-side comparison of the performance benchmarks. For convenience, Accelbench automatically highlights the “hero” metrics, i.e. the best metrics in each category, giving users a visual indicator which instance type provided the best performance. A summary at the top of the report attempts to summarize the results of the benchmarks.

To create a comparison report, select multiple benchmarks from the **Benchmarks** page, then click on the **Compare** button. You can compare up to 4 benchmark runs at once.



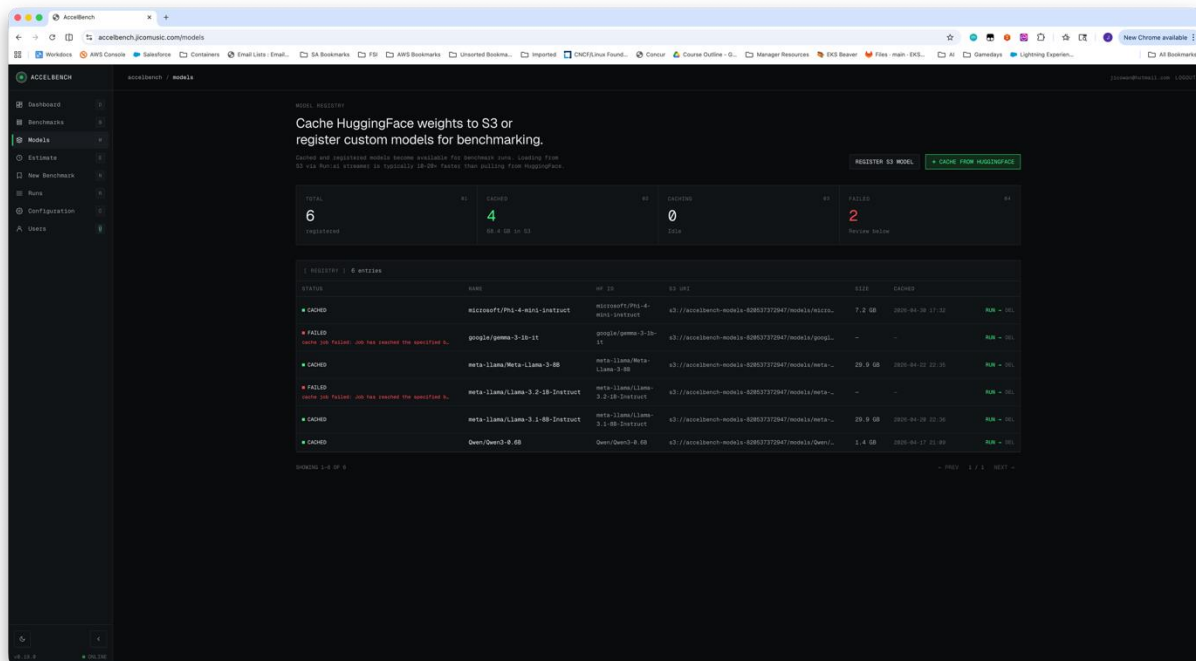
Seeding

Rather than running one-off benchmarks, Accelbench includes a feature that allows you to run a series of benchmarks all at once. The specific instance types and models included in the seeding run need to be configured by an administrator. A seeding run can be initiated by clicking the **Seeding** button on the **Benchmarks** screen.

Models

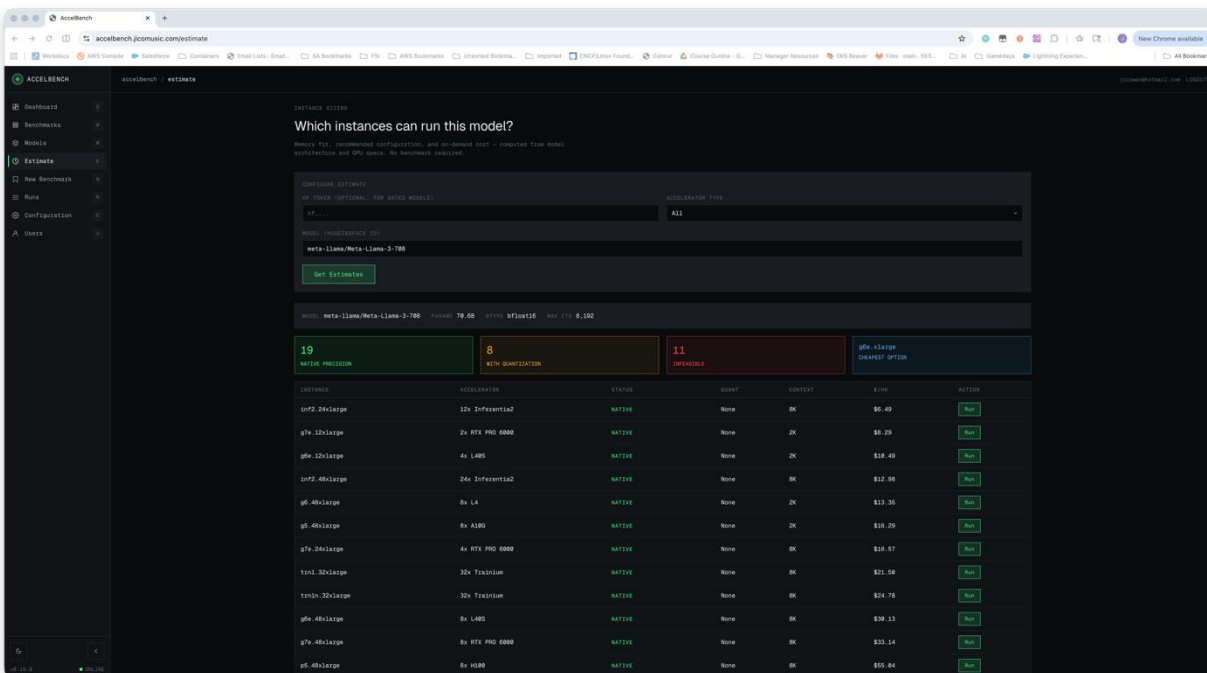
From the **Models** page, you can register a bespoke/fine-tuned open-source model that is stored on S3 or you can cache a model from Huggingface to S3. Caching the model weights to S3 can improve model load times because S3 is not subject to Huggingface's API limits. Additionally, when you cache a model to S3, Accelbench uses run.ai's model streamer which can shorten model load times even further.

Note: if you register a model stored on S3, you will need to grant the IAM role, accelbench-model, the ability to read from that bucket/prefix.



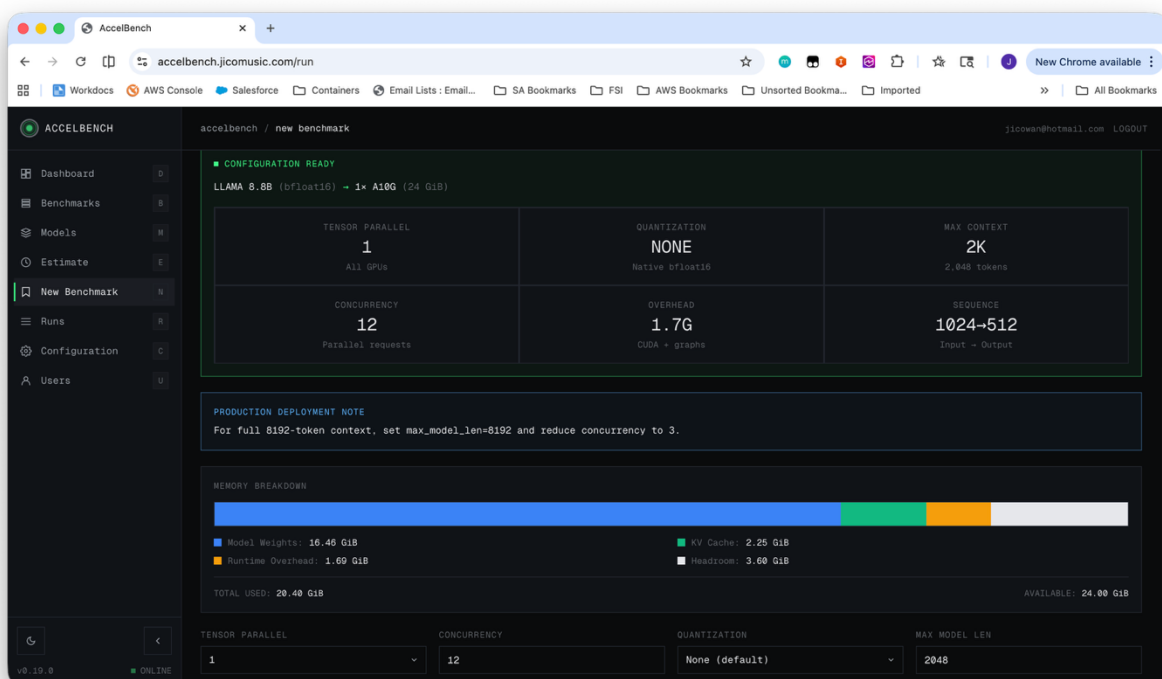
Estimate

The **Estimate** page reveals the EC2 instance types that are capable of running a particular model. The results show which instance types can run the model with native precision, and which instance types can run the model with quantization. The cheapest option is highlighted too. Clicking the **Run** button for a model will pre-populate the **Run Benchmark** form with the recommended parameters for running a benchmark for that model.



New Benchmark

The **New Benchmark** page is for configuring and running new benchmarks. From this page, users can select the model and instance type they want to benchmark. By default, the platform will automatically set the model's runtime parameters based on the model's metadata; however, these parameters can be overridden if necessary. The benchmark type (scenario or suite) can be configured from this page too. Currently, Accelbench uses inference-perf for benchmarking models. With Accelbench and inference-perf, you can use real-world datasets such as ShareGPT, CNN Dailymail, Billsum, and Infinity Instruct, to send a variety of different prompts to the model. The **Scenario** drop down controls the behavior of inference-perf, allowing you to choose which scenario you want to use to benchmark the model (chatbot, batch processing, stress test, stress test, and production traffic), whereas the **Suite** option allows you to benchmark a combination of individual scenarios.



The built-in recommendation engine automatically generates model runtime parameters for vLLM, the model runtime used by AccelBench. In the **Configuration Ready** section, users can see an explanation of each of the runtime parameters and the engine's recommendation based on the model's metadata. If a run fails because the memory overhead is set too low, users can override the parameters or use the **Runtime Overhead** slider to adjust the runtime parameters.

The **Run Benchmark** page also generates a graph of the memory breakdown for the model. Ideally, the memory breakdown will include enough headroom for the model to run it without issue. If you see a model has no headroom, it is unlikely to run on the chosen instance type.

Note: the raw results of a benchmark are stored in an S3 bucket before they're written to the database. If you need to access the raw results, navigate to `s3://accelbench-results-{AWS_ACCOUNT_NUMBER}/results bucket/prefix`.

When you are finished configuring the benchmark's parameters, click the **Start Benchmark** button to run the benchmark. This causes AccelBench to create a Kubernetes Deployment that encapsulates all the runtime parameters along with a node selector for the instance type that was chosen. The node selector informs Karpenter which instance type to provision. When the model is ready to start serving traffic, the AccelBench orchestrator will launch an instance of inference-perf which places a load on the model and collects performance metrics. Before the inference-perf container is terminated, the results are written to the aforementioned S3 bucket. The AccelBench orchestrator reads these results and writes them, together with hardware metrics it collected during the run, to the database.

Runs

After starting a new benchmark, the run will appear on the Runs page. The benchmark report for the run will not be available until the benchmark is completed. If there is a failure during the run, the report will indicate the reason the run failed.

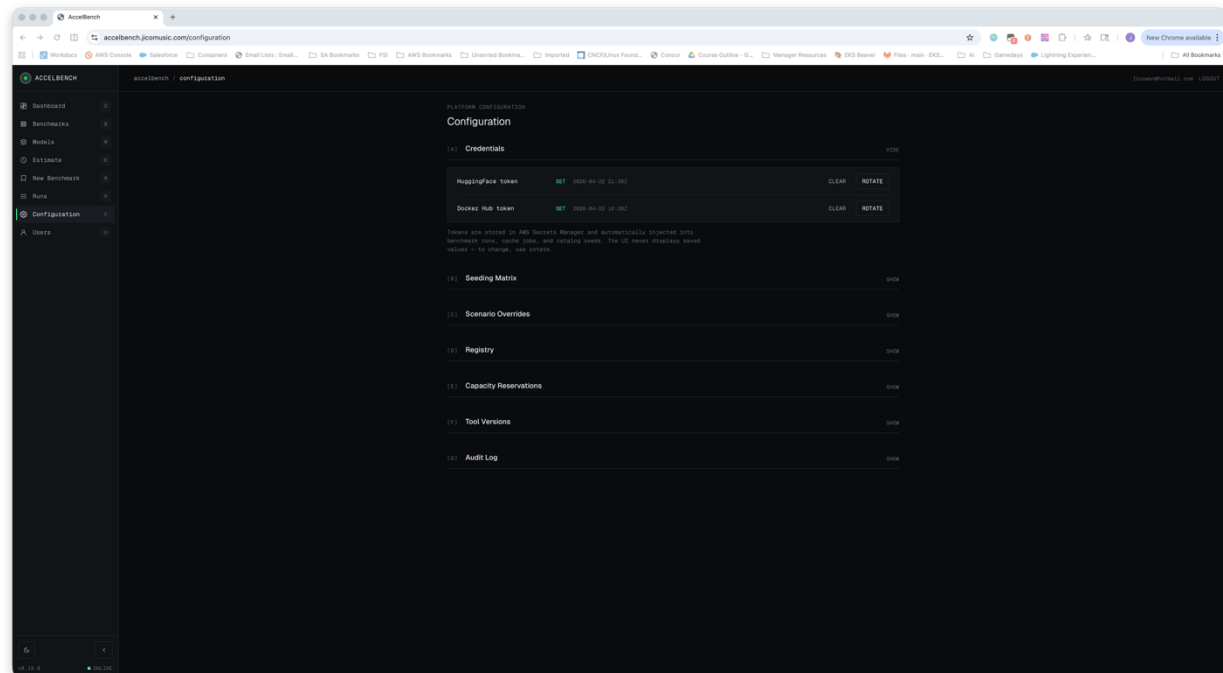
Note: It may be necessary to run `kubectl logs` against the vLLM pod to diagnose repeated failures or to collect additional troubleshooting information.

Benchmark runs configured to use the **Suite** option where multiple scenarios are run against a model/instance only will appear on the **Runs** page.

While all runs, regardless of **Benchmark Type** or status, will appear on the **Runs** page; you can only compare runs from the **Benchmark** catalog page.

Admin guide

The Admin section of the interface is divided into 2 parts: Configuration and Users. These sections are restricted to users assigned the Admin role which we'll explore shortly.



The Configuration page is where an Accelbench administrator can affect the global settings of the platform. The page is separated into multiple sections. Let's start by looking at the **Credentials** section.

Credentials

The **Credentials** page is used for storing the default tokens accessing Dockerhub and Huggingface. If the default Huggingface token is not authorized to access a particular model, users still possess the ability to override the default token with their own token on the **Estimates**, **Models**, and **Run Benchmark** forms.

The official vLLM image that AccelBench uses is stored in Dockerhub which is subject to throttling and API limits. Given the image's size and how often it will need to be pulled, we recommend creating an ECR pull-through cache for the vLLM image. AccelBench uses the Dockerhub token to create that pull-through cache. Once the pull-through cache is enabled, AccelBench will pull the vLLM image from ECR rather than Dockerhub

which should improve image pull times considerably, especially on larger instances that benefit from SOCI parallel pulls.

The tokens for both Huggingface and Dockerhub are stored in AWS Secrets Manager. Once a token is entered into AccelBench it cannot be viewed from within the UI. If you need to update the token, you can do so by clicking the **Rotate** button.

Seeding Matrix

The **Seeding Matrix** section is used for controlling the default settings for the **Seed Benchmarks** function. The **Seeding Matrix** lets you specify which inference-perf scenario and dataset will be used for benchmarking the selected models and instance types. When users click the **Seed Benchmarks** button from the **Benchmarks** page, AccelBench will dynamically compile a matrix of benchmarks to run on the platform. When doing so, it uses the recommendation engine to assess the viability of running a model on a particular instance before including it in the matrix. Non-viable combinations of model/instant type will be omitted. It will also omit duplicates, e.g. if you've previously benchmarked a particular combination, it will be excluded from the matrix.

Scenario Overrides

The **Scenario Overrides** section is used for overriding the default settings for each inference-perf scenario. For each scenario, you can configure the number of workers, whether streaming is on/off, and the input and output mean. The number of workers should correspond to the number of cores available to the inference-perf pod. When streaming is set to `on`, it enables streaming for TTFT, ITL, and TPOT metrics. Input and output mean control the average length of the prompts sent to the model.

Registry

The **Registry** section indicates whether the ECR pull-through cache for vLLM has been created.

Capacity Reservations

Capacity Reservations allows you to associate an On-demand Capacity Reservation (ODCR) or Capacity Block (CB) reservation ID with a Carpenter node pool.

Note: Currently, the reservation has to be for the instance types and availability zones specified in the 2 Karpenter node pools. If you used the Terraform that was included in the AccelBench repository to bootstrap AccelBench, 2 node pools get created automatically — gpu and neuron — each with specific instance types.

ODCRs and CBs are a discounting and consumption mechanism that provides users access to specific instance types (GPUs) for specific periods of time. They're often used in conjunction with model training and other workloads that require specialized instance types that are in high demand.

Tool Versions

The Tool Version section is used to control which version of vLLM and inference-perf is used. When a new version of vLLM or inference-perf is released, an administrator can use this section to specify which version (image tag) AccelBench should use for running models and benchmarks.

Audit Log

The **Audit Log** shows a sliding window of the last 50 configuration changes.

Users

The **Users** section is for user administration. From this page, administrators can invite new users to the platform, reset user passwords, disable/enable user accounts, or change a user's assigned role.

Users can be assigned 1 of 2 roles: user or admin. A user can access all of the sections described in the end-user guide. An admin has access to all the sections in the admin guide in addition to all the sections in the end-user guide.

Quickstart

The quickest way to get started with Accelbench is by navigating to the **Estimate** page and entering the model you want to benchmark. When the list of instance types appears, click the **Run** button on the row for the instance type you want to use. This will populate the **Run Benchmark** form with the recommended vLLM parameters for that model. Choose the scenario/dataset you want to use. Once you are satisfied with the settings,

click the **Start Benchmark** button. The length of time you'll need to wait before seeing results will depend on the size of the model, the instance type it is running on, whether the model weights are cached on S3, and whether the vLLM image is being pulled from the pull-through cache.

Reporting bugs

AccelBench is an aws-samples project and may still have bugs. If you encounter a bug while benchmarking a model, please open an issue in its GitHub repository, <https://github.com/aws-samples/sample-eks-model-benchmarks>. We will try to resolve issues as quickly as possible.